JAVA / 阶段 02

面向对象程序设计

阶段二 · 零基础详解

对应原题库全部 17 道题

从基础语法到独立完成程序

使用方式

每道题保存为独立的 Main.java。先读题并自己尝试，再对照答案运行；不要把不同题的 Main 类放进同一个文件。

命令：javac -encoding UTF-8 Main.java；随后执行 java Main。输入样例中的数据后按回车。网站可直接编辑、编译和运行。

解法使用变量、判断、循环、数组、方法和基础面向对象知识，不依赖第三方 Java 库。

题目口径与说明

1. 依据用户提供的阶段二题库 PDF 整理，共17道独立编程题。原题前7题后又重复编号6、7，本资料按出现顺序编号1至17，并保留原题号。

2. 采用中国大学二年级常见基础写法：普通类、字段、构造、getter/setter、if、for、数组、继承、接口与抽象类；不使用Stream、Lambda或第三方库。

3. 每题均有独立public class Main；其余顶层类和接口不加public，保存为Main.java即可。不同题中的同名类必须分开编译运行。

4. 图形绘制题和狮子题已对照原 PDF 类图及运行截图，字段、参数与继承关系按原图整理。

5. 明确约定：Point返回距离平方；同时提供无参/有参构造；银行账户无透支，最终余额1500元；Vehicle的start/stop按文字要求各带一个String参数。

6. Circle/Cylinder区分底面积与表面积，并使用super.getArea防止多态导致体积公式错误；CanCry保留G类及G.main，由Main启动。

目录

学习起点：先认识 Java 程序 3

第 01 题 三维点 Point 与到原点的距离平方 5

第 02 题 长方形的面积、周长与 toString 7

第 03 题 学生类的访问控制与两种构造 9

第 04 题 银行账户存款、取款与余额检查 11

第 05 题 手机类与两种对象初始化方式 13

第 06 题 学生阅读图书：对象作为参数 15

第 07 题 Person：姓名校验、固定血型与身高不减 17

第 08 题 People 基类、Employee 与 Teacher 子类 19

第 09 题 Animal、Cat、Bird 与运行时多态 21

第 10 题 Vehicle 接口、Bike/Bus 与 Driver 23

第 11 题 Circle 与 Cylinder：继承、面积重写和体积 25

第 12 题 CanCry 接口与 G.makeCry 测试 28

第 13 题 抽象 Shape 与圆、矩形面积 29

第 14 题 员工数组、平均工资项目与应发工资 31

第 15 题 Graph 类图：三种图形与 Client 绘制 34

第 16 题 Animal、Ibehavior 与 Lion 类图实现 36

第 17 题 网络用户类与默认邮箱构造重载 38

复习与参考 40

学习起点：先认识 Java 程序

推荐顺序：先运行最小程序，再读概念，再手写题目。每次只改一小处，编译后观察结果；报错时从第一条错误开始看。

最小可运行程序

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello Java");
    }
}
```

Main 是类名，必须与 Main.java 的文件名一致。main 是程序入口，花括号决定代码所属范围；println 表示输出一行，英文分号结束一条语句。

本地练习：安装 JDK 17，在 Main.java 所在目录运行 javac -encoding UTF-8 Main.java，再运行 java Main。javac 负责检查并编译，java 执行生成的字节码。编译失败时不会产生新的正确程序。

网站练习：选一道题，在编辑区输入代码，在标准输入区填数据，点击“编译并运行”。先观察编译状态，再读程序输出；“样例期望输出”仅用于对照。

1. 0. 先学会运行一题：输入、程序与输出

每题代码都独立保存为 Main.java，文件名与 public class Main 完全一致。先安装 JDK 17，打开文件所在目录的终端，执行 javac -encoding UTF-8 Main.java 编译，再执行 java -Dfile.encoding=UTF-8 Main 运行。javac 把人可读的源代码变为 .class 字节码，java 才执行它；只编译成功不会自动产生题目输出。阶段二只有长方形题和动物选择题需要标准输入，其他题使用 main 中的演示数据。输入框只填数据，不把“请输入”等提示文字填进去。每道题单独目录运行，因为不同题可重复定义 Student、Circle 等类。

```
javac -encoding UTF-8 Main.java
java -Dfile.encoding=UTF-8 Main
```

2. 1. 读懂 Java 程序的外壳

public class Main 定义一个公开类。public static void main(String[] args) 是程序入口：运行 java Main 后从其中第一句开始按顺序执行；void 表示没有返回值，static 使入口不必先 new Main。花括号包住类、方法、分支的范围，大多数赋值、调用语句以分号结尾。// 后面到行末是注释，帮助人阅读，不会执行。System.out.println(...) 输出一行，System.out.printf(...) 可按指定格式输出。暂时记住 main 的固定写法，再把精力放在其中语句如何改变数据。

3. 2. 变量、类型、运算与条件

int 保存整数，如年龄18；double 保存小数，如身高1.75；String 保存双引号中的文字；boolean 保存 true 或 false。变量先声明后使用，int age = 18 同时声明并赋值。= 是赋值，== 才是比较；+、-、*、/ 分别加减乘除；两个整数相除会舍去小数。字符串参与 + 时会连接文字。if (条件) { ... } 在条件为真时执行，else 处理另一种情况。|| 是“或”，&& 是“且”；它们短路求值。return 把值交给调用者或结束 void 方法。先掌握这些，就能读银行余额和姓名校验。

4. 3. 类、对象、引用：把数据和行为放在一起

类是一种类型或图纸，对象是按图纸建立的一个具体实例。Phone 表示手机这种类型，new Phone("华为",998) 创建一部具体手机。Phone p 中 p 是引用变量，保存访问对象所需的引用值；p.call() 表示让这个对象执行行为。成员变量写在类里、方法外，每个普通对象各有一份；局部变量写在方法内，只能在自己的作用域使用。普通成员方法没有 static，需要通过对象调用。对象参数传递时复制的是引用值，因此两个引用可能访问同一对象，Java始终按值传参。

```
Phone p = new Phone("华为", 998);
p.call();
```

5. 4. 封装、getter、setter 与 this

封装把内部数据藏起来，只通过允许的方法访问。private 字段主要由本类内部使用，public 方法供外部调用；protected 还允许同包和符合规则的子类访问。getAge 通常返回年龄，setAge 通常接收新年龄并写入；它们不是Java关键字，只是常见命名。this 表示当前对象，因此 this.age = age 左边访问对象字段，右边读取同名形参。可以在setter赋值前检查范围，非法时拒绝更新。final 字段只允许赋值一次，Person 的血型正适合这个做法。

```
private int age;
public int getAge() { return age; }
public void setAge(int age) { this.age = age; }
```

6. 5. 构造方法与重载：对象出生时就有数据

构造方法和类同名，没有返回类型，在new时自动调用。无参构造不接收数据，有参构造接收初始化信息。方法名相同而参数列表不同叫重载，因此能同时提供 Point() 和 Point(double x,double y,double z)。只有当类完全没声明构造时，编译器才自动提供无参构造；自己写了有参构造后，需要无参版就必须再写。this(...) 调用本类其他构造并必须放第一句，可避免重复赋值。网络用户用两个构造实现“邮箱可省略”，这是Java常见默认值处理方式。

7. 6. 继承、super 与构造顺序

class Cat extends Animal 表示猫是一种动物，Cat复用Animal中可继承的字段和方法，并增加自己的毛色。构造方法不会被继承，创建子类时先完成父类初始化再初始化子类。super(name) 调用父类构造，必须在子类构造第一句；如果不写，Java尝试插入super()，但父类没有无参构造时会编译失败。super.getArea() 指定调用父类的面积实现，this.getArea() 则仍会参与多态调用。不要混淆 super(...) 构造调用与 super.方法名(...) 普通父类方法调用。

8. 7. 重写与多态：声明类型和实际类型可以不同

重写是子类提供父类同名同参数方法的新实现，例如 Cat.sing()；@Override 让编译器帮忙核对。多态允许 Animal pet = new Cat(...)，左边的声明类型决定编译时能调用哪些方法，右边实际对象决定运行时采用哪个可重写实例方法实现。Animal引用调用sing会发出猫叫。重载看参数列表来区分，重写改变继承方法的实现，二者不同。圆柱的 getArea 重写为表面积，getVolume因此必须明确用super.getArea求底面积，不能把动态派发后的getArea当作圆面积。

9. 8. 接口和抽象类：先约定行为，再给出实现

interface CanCry 约定cry行为；Dog implements CanCry 必须提供public cry实现。接口不能直接new，但接口类型可声明参数或引用，makeCry(CanCry c)因此能接收狗和猫。abstract class Shape是抽象类，public abstract double getArea();规定子类必须实现面积方法；抽象类也不能直接实例化。Java类只能直接继承一个类，但能实现多个接口，例如Lion extends Animal implements Ibehavior。抽象类常表示共同的“是什么”，接口常表示共同的“能做什么”。本题不需要更复杂的接口默认方法。

10. 9. 数组、循环、平均数与输出格式

Employee[] employees = {...}把多个员工引用放进数组，数组长度用employees.length读取，下标从0到length-1。for(int i=0; i<employees.length; i++)从第0个遍历到最后一个。累加器先设0，在循环里加每个对象的字段，最后再除以人数；double累加器确保平均数保留小数。printf的%s输出字符串、%d输出整数、%.2f输出保留两位小数的浮点显示、%n换行。舍入发生在显示时，计算仍使用原始double。Locale.ROOT让示例统一使用小数点。

11. 10. 如何用这17题从看懂到会写

先运行一题并核对输出，再逐行解释代码，之后关掉答案尝试重写。推荐先做1～6题练对象和封装，再做7题练校验，再做8～13题练继承、接口和多态，随后做14题数组统计，最后做15～17题综合练习。每题至少改一个数据并先手算输出，避免只记原答案。遇到编译错误，先看第一条错误的行号，检查分号、花括号、类型、名字大小写和文件名；程序能运行但数值不对，再检查公式与更新顺序。try/catch只用于演示可预期的非法调用，不应该用它掩盖自己没理解的错误。

如何判断自己学会了

合上答案，能说出输入、处理过程和输出；能自己写出主要代码；能解释循环每轮变量怎么变化；能修改样例并预测结果。只看懂代码不等于会写，至少独立重写一次。

第 01 题 三维点 Point 与到原点的距离平方

原题编号：1；本书按出现顺序编为第 1 题。

题目

定义 Point 类，用浮点型成员 x、y、z 表示三维坐标。提供构造方法、setX、setY、setZ，getDistance() 返回点到原点的距离平方。原题写 Point() 又要求特定坐标，因此同时提供无参构造 Point() 和有参构造 Point(double x, double y, double z)，并实例化验证。

基础思路

用 double 保存坐标。无参构造创建原点，有参构造直接接收坐标；三个 setter 分别改坐标，距离平方为 x*x+y*y+z*z。

先掌握这些知识

类与对象、成员变量、构造方法重载、this、平方和

一步一步理解

1. class Point 是图纸，new Point() 才真正创建一个点；p1 保存这个对象的引用。

2. private double x 表示每个 Point 对象各自保存一个小数坐标，外部不能直接写 p1.x。

3. 构造方法名称必须与类名一致，没有 void 等返回类型。Point() 没有参数，Point(double x, double y, double z) 有三个参数，这叫重载。

4. this(0, 0, 0) 调用本类的另一个构造方法，必须写在当前构造方法第一条语句的位置。

5. this.x = x 中左边是当前对象的字段，右边是传进来的参数。setX(3) 就把该对象的 x 改成 3。

6. 三维空间中距离的平方等于 $x^2+y^2+z^2$，(1,2,3) 得 1+4+9=14；题目没有要求开平方。

7. main 先验证无参构造，再验证有参构造，最后同时验证三个 setter；double 输出会带 .0。

完整参考代码 · Main.java

```
class Point {
    private double x;
    private double y;
    private double z;

    public Point() {
        this(0, 0, 0);
    }

    public Point(double x, double y, double z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }

    public void setX(double x) { this.x = x; }
    public void setY(double y) { this.y = y; }
    public void setZ(double z) { this.z = z; }

    public double getDistance() {
        return x * x + y * y + z * z;
    }
}

public class Main {
    public static void main(String[] args) {
        Point p1 = new Point();
        System.out.println("原点的距离平方：" + p1.getDistance());
        Point p2 = new Point(1, 2, 3);
        System.out.println("点(1,2,3)的距离平方：" + p2.getDistance());
        p1.setX(3);
        p1.setY(4);
        p1.setZ(0);
        System.out.println("点(3,4,0)的距离平方：" + p1.getDistance());
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
原点的距离平方：0.0
点(1,2,3)的距离平方：14.0
点(3,4,0)的距离平方：25.0
```

容易出错的地方

1. Java 的 ^ 是按位异或，不是乘方；平方直接写 x * x。

2. getDistance 的题意是距离平方，不能添加 Math.sqrt。

3. 只写有参构造后，Java 不再自动提供无参构造，因此两种构造都要显式写。

自己动手

1. 把第二个点改为 (-1, -2, -3)，验证距离平方仍为 14。

2. 给 Point 增加三个 getter，在 main 输出修改后的坐标。

第 02 题 长方形的面积、周长与 toString

原题编号：2；本书按出现顺序编为第 2 题。

题目

设计长方形类，成员为长 length、宽 width；构造方法初始化两个字段；getArea() 求面积，getPerimeter() 求周长；重写 Object.toString()。测试类从标准输入依次读取长、宽并创建对象，通过 toString 输出长、宽、面积、周长。输入 10 和 20 时输出“这个长方形长为10，宽为20，面积为200，周长为60。”。本基础示例按原示例采用正整数边长。

基础思路

用 int 保存示例中的整数边长，面积为 length*width，周长为 2*(length+width)，把完整说明放入 toString。

先掌握这些知识

Scanner 输入、构造方法、返回值、toString 重写、字符串拼接

一步一步理解

1. import java.util.Scanner 引入读取键盘输入的工具；new Scanner(System.in) 表示从标准输入读取。

2. 两个 nextInt() 依次取整数，换行或空格都可分隔，输入顺序分别对应长和宽。

3. new Rectangle(length, width) 调用构造方法，将 main 中两个局部变量的值复制到新对象字段。

4. getArea 返回两个边长的乘积，getPerimeter 先计算括号内两边之和，再乘 2。return 会把结果交给调用者。

5. 所有类都继承 Object；重写 toString 可以把对象变成有意义的文字，方法返回类型是 String。@Override 请编译器检查是否确实重写。

6. toString 内用 + 把文字和数值连接；遇到字符串后，后面的 + 用于连接文字。这里面积和周长先由方法计算。

7. 也可以写 System.out.println(r)，println 会自动使用该对象的 toString。

完整参考代码 · Main.java

```
import java.util.Scanner;

class Rectangle {
    private int length;
    private int width;

    public Rectangle(int length, int width) {
        this.length = length;
        this.width = width;
    }

    public int getArea() { return length * width; }
    public int getPerimeter() { return 2 * (length + width); }

    @Override
    public String toString() {
        return "这个长方形长为" + length + "，宽为" + width
            + "，面积为" + getArea() + "，周长为" + getPerimeter() + "。";
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int length = sc.nextInt();
        int width = sc.nextInt();
        Rectangle r = new Rectangle(length, width);
        System.out.println(r.toString());
    }
}
```

样例输入

```
10
20
```

样例输出

```
这个长方形长为10，宽为20，面积为200，周长为60。
```

容易出错的地方

1. toString 名称的 S 必须大写，不能写 toString(String s)。

2. 周长需要括号，2 * length + width 会少算一条宽。

3. int 适合本题小整数练习；大数或小数边长需要改类型，当前练习未处理非数字输入。

自己动手

1. 输入 3 和 4，先手算再核对面积 12、周长 14。

2. 把字段、构造参数和输入改成 double，观察输出格式的变化。

第 03 题 学生类的访问控制与两种构造

原题编号：3；本书按出现顺序编为第 3 题。

题目

设计 Student 类，保存姓名、年龄、身高、年级：姓名为 public 字段，其他字段为 private；年龄为读写属性，使用 getAge()/setAge() 实现。提供无参和全参构造方法并实例化测试。为展示私有字段，增加 showInfo() 方法。

基础思路

按要求区分字段权限，用 getter 和 setter 公开年龄读写；通过两个构造方法展示默认对象和完整初始化对象。

先掌握这些知识

public/private、getter/setter、无参构造、有参构造、封装

一步一步理解

1. name 是 String，年龄是 int，身高是 double，年级用 String 保存“大一”等文字。身高单位在本题统一为米。

2. public name 允许 main 直接执行 s1.name = "张三"。private age 则只能由 Student 类中的方法访问。

3. Java 没有此题所说的属性语法，通常使用 getAge 读、setAge 写；这两个普通方法组成了年龄的读写接口。

4. 无参构造里的默认姓名、18 岁、1.60 米、大一是演示设定，原题没有规定默认值；也可以选择其他合理默认值。

5. 有参构造按姓名、年龄、身高、年级的顺序接收参数，因此 new Student("李四", 20, 1.75, "大二") 直接建立完整记录。

6. showInfo 在 Student 类内部，可以直接读取 private 字段；main 通过公开方法查看内容，不需要把字段改成 public。

7. 本题聚焦访问控制和构造；若用于实际登记，可在 setAge 中加入非负检查，并在构造中调用它。

完整参考代码 · Main.java

```
class Student {
    public String name;
    private int age;
    private double height;
    private String grade;

    public Student() {
        this("未命名", 18, 1.60, "大一");
    }
```

Main.java（续）

```
    public Student(String name, int age, double height, String grade) {
        this.name = name;
        this.age = age;
        this.height = height;
        this.grade = grade;
    }

    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }

    public void showInfo() {
        System.out.println("姓名：" + name + "，年龄：" + age
            + "，身高：" + height + "米，年级：" + grade);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.name = "张三";
        s1.setAge(19);
        s1.showInfo();
        Student s2 = new Student("李四", 20, 1.75, "大二");
        s2.showInfo();
        System.out.println("李四的年龄：" + s2.getAge());
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
姓名：张三，年龄：19，身高：1.6米，年级：大一
姓名：李四，年龄：20，身高：1.75米，年级：大二
李四的年龄：20
```

容易出错的地方

1. 不要在 Student 外直接写 s1.age。
2. 构造方法不能声明 void，否则只是同名普通方法。
3. 题目要求姓名公有，即使平常更常用 private，此处也应按题意声明 public。

自己动手

1. 再创建一名大二学生并输出资料。
2. 尝试把 s1.age = 19 放在 main 中，读懂编译错误后恢复 setAge。

第 04 题 银行账户存款、取款与余额检查

原题编号：4；本书按出现顺序编为第 4 题。

题目

创建 BankAccount 类，私有字段为账号和余额，为两字段提供 setter/getter，提供全参构造和存款、取款、查询余额方法。在 main 创建 ba，账号 123456、余额 500 元，存入 1000 元，尝试取出 2000 元，最后查询余额。这里采用普通无透支账户规则：余额不足时拒绝取款，因此最终余额 1500 元。

基础思路

账号用 String，余额用 double。操作先验证金额，再判断余额；取款失败时 return，不改变原余额。

先掌握这些知识

封装、条件分支、提前 return、状态变化、方法调用

一步一步理解

1. 账号属于标识符，不做加减，用 String 可保留将来可能出现的前导零；余额参与计算，用 double。

2. 全参构造调用 setAccount 和 setBalance，建立对象时也执行余额不能为负的检查。IllegalArgumentException 表示调用者给出的参数不合要求。

3. deposit(1000) 把 1000 传到局部参数 money。balance += money 等价于 balance = balance + money，所以余额从 500 变成 1500。

4. withdraw(2000) 先判断 money 是否有效，再比较 2000 和当前余额 1500。条件成立后打印失败信息并 return。

5. return 在 void 方法中表示立即结束本次调用，所以 balance -= money 不会执行，余额仍为 1500。

6. queryBalance 打印账号与余额，getBalance 则把余额作为返回值交给其他代码，两种查询方式都演示了公开接口。

7. 此题未规定透支政策，答案明确使用无透支规则；不能不加说明就把余额算成 -500。double 用于基础课堂演示，真实财务系统通常需使用精确金额类型。

完整参考代码 · Main.java

```
class BankAccount {
    private String account;
    private double balance;

    public BankAccount(String account, double balance) {
        setAccount(account);
        setBalance(balance);
    }

    public String getAccount() { return account; }
    public void setAccount(String account) { this.account = account; }
    public double getBalance() { return balance; }
    public void setBalance(double balance) {
        if (balance < 0) {
```

Main.java（续）

```
            throw new IllegalArgumentException("余额不能为负数");
        }
        this.balance = balance;
    }

    public void deposit(double money) {
        if (money <= 0) {
            System.out.println("存款金额必须大于0");
            return;
        }
        balance += money;
        System.out.println("存款成功：" + money + "元");
    }

    public void withdraw(double money) {
        if (money <= 0) {
            System.out.println("取款金额必须大于0");
            return;
        }
        if (money > balance) {
            System.out.println("余额不足，取款失败");
            return;
        }
        balance -= money;
        System.out.println("取款成功：" + money + "元");
    }

    public void queryBalance() {
        System.out.println("账号：" + account + "，余额：" + balance + "元");
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount ba = new BankAccount("123456", 500);
        ba.deposit(1000);
        ba.withdraw(2000);
        ba.queryBalance();
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
存款成功：1000.0元
余额不足，取款失败
账号：123456，余额：1500.0元
```

容易出错的地方

1. 只输出“余额不足”却继续执行减法仍会造成负余额，必须 return 或使用 else。

2. 存款传负数会等同取款，所以也需要检查存款金额。

3. 不要在 main 中直接访问 private balance。

自己动手

1. 把取款改为 1200，验证最终余额 300。

2. 在成功存款后依次尝试取出 0 元、1500 元，观察失败和清零的差别。

第 05 题 手机类与两种对象初始化方式

原题编号：5；本书按出现顺序编为第 5 题。

题目

创建 Phone 类，私有字段为品牌和价格，提供两字段的 setter/getter、无参和全参构造，以及打电话、听歌两个方法。在 main 中分别用 setter 和全参构造创建两部手机，输出“正在使用价格为998元的华为手机打电话”和“正在使用价格为998元的小米手机听歌”。空格仅为排版，本答案使用无空格的中文行。

基础思路

先 new Phone() 再设置华为手机信息；小米手机直接调用全参构造。行为方法读取各自对象的品牌和价格。

先掌握这些知识

对象初始化、封装、构造重载、实例方法

一步一步理解

1. Phone 类把数据 brand、price 和行为 call、listenMusic 放在一起，这就是用对象描述一部手机。

2. 无参构造方法体为空也合法。对象创建后，String 字段默认 null，int 字段默认 0，因此必须先设置再使用。

3. p1.setBrand("华为") 和 p1.setPrice(998) 分两次写入属性，演示通过 setter 完成初始化。

4. new Phone("小米", 998) 一次传入全部属性，由有参构造完成初始化。

5. p1 和 p2 指向两部不同的手机；调用 p1.call() 时，call 使用 p1 对象里的数据。

6. 价格示例是整数，所以用 int 可直接输出 998；若需要 998.50 元，应改为合适的小数类型并统一显示格式。

完整参考代码 · Main.java

```
class Phone {
    private String brand;
    private int price;

    public Phone() { }

    public Phone(String brand, int price) {
        this.brand = brand;
        this.price = price;
    }

    public String getBrand() { return brand; }
```

Main.java（续）

```
    public void setBrand(String brand) { this.brand = brand; }
    public int getPrice() { return price; }
    public void setPrice(int price) { this.price = price; }

    public void call() {
        System.out.println("正在使用价格为" + price + "元的" + brand + "手机打电话");
    }

    public void listenMusic() {
        System.out.println("正在使用价格为" + price + "元的" + brand + "手机听歌");
    }
}

public class Main {
    public static void main(String[] args) {
        Phone p1 = new Phone();
        p1.setBrand("华为");
        p1.setPrice(998);
        Phone p2 = new Phone("小米", 998);
        p1.call();
        p2.listenMusic();
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
正在使用价格为998元的华为手机打电话
正在使用价格为998元的小米手机听歌
```

容易出错的地方

1. 有参构造存在时不会自动生成无参构造，需要自己声明 Phone()。
2. 方法调用必须有小括号，例如 p1.call()。
3. 不要把两个品牌都存成 static 字段，否则就不再是每部手机各自的品牌。

自己动手

1. 使用 getter 输出 p2 的品牌和价格。
2. 把第二部手机改成其他品牌并调用两种行为，观察对象之间是否互相影响。

第 06 题 学生阅读图书：对象作为参数

原题编号：6；本书按出现顺序编为第 6 题。

题目

创建 Book 类，私有属性为作者和书名；创建 Student 类，私有属性为姓名和年龄，提供一个读书方法，所读图书由参数传入。两类均提供所有属性的 setter/getter 和全参构造。在 main 创建一名18岁的张三和《资治通鉴》图书对象，调用读书方法，输出“18岁的张三正在读的书是《资治通鉴》”。

基础思路

Book 单独管理图书信息；Student.readBook(Book book) 通过传入图书的 getTitle() 获取书名并打印。

先掌握这些知识

对象作为参数、类之间的协作、getter、构造方法

一步一步理解

1. Book 和 Student 是两种对象：一本书保存作者、书名，一名学生保存姓名、年龄。这样无需把书的信息混进学生字段。

2. new Book("司马光", "资治通鉴") 参数顺序对应作者、书名；参数顺序一致才能获得正确输出。

3. readBook(Book book) 中 Book 是参数类型，book 是参数名。它表明这个方法需要一本书对象，而不是仅需要书名字符串。

4. student.readBook(book) 把已有图书对象的引用值传入方法；Java 始终按值传参，这里复制的是引用值，没有自动复制整本书。

5. book.getTitle() 通过公开方法读取 Book 的 private title，体现两个类之间通过方法协作。

6. 输出中的年龄、姓名来自当前 Student，书名来自参数 Book，因此同一学生可以在不同调用中阅读不同书。

完整参考代码 · Main.java

```
class Book {
    private String author;
    private String title;

    public Book(String author, String title) {
        this.author = author;
        this.title = title;
    }

    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
}

class Student {
    private String name;
```

Main.java（续）

```
    private int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }

    public void readBook(Book book) {
        System.out.println(age + "岁的" + name + "正在读的书是《"
            + book.getTitle() + "》");
    }
}

public class Main {
    public static void main(String[] args) {
        Book book = new Book("司马光", "资治通鉴");
        Student student = new Student("张三", 18);
        student.readBook(book);
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
18岁的张三正在读的书是《资治通鉴》
```

容易出错的地方

1. Student 不能直接访问 Book 的 private title，应调用 getTitle()。
2. 不要在 readBook 内把书名固定写死，否则传入其他书仍输出原书名。
3. 这里调用前已创建对象；如果传入 null，再调用 getTitle() 会出现空指针异常。

自己动手

1. 再创建一本其他书，让张三再次阅读。
2. 把 readBook 输出扩展为同时显示作者，练习调用 book.getAuthor()。

第 07 题 Person：姓名校验、固定血型与身高不减

原题编号：7；本书按出现顺序编为第 7 题。

题目

设计 Person 类：姓名 private，血型 private 且创建后不可修改，另有身高字段。提供构造方法、getName、setName 和 addHeight；姓名必须为2～4个字符，身高不能减少。main 创建对象，并验证正常改名、非法改名、增加身高与负增长被拒绝。为读取结果补充 getBloodType 和 getHeight。

基础思路

血型声明 private final 且只在构造中赋值，不提供 setter。姓名统一在 setName 校验，addHeight 只接受非负增量。

先掌握这些知识

private final、数据校验、异常处理、不可变字段、累加

一步一步理解

1. private 使外部不能直接修改字段；final 要求 bloodType 只能赋值一次，且这里没有血型 setter，创建后就固定。

2. 构造方法先检查初始身高非负，再调用 setName，保证初始姓名同样满足2～4个字符。

3. name == null 先判断是否没有字符串；|| 是短路或，前面成立时后面不再执行，因此不会对 null 调用 length()。

4. 常见汉字和英文字母可用 String.length() 计数。本基础题按 Java 字符单元长度处理；表情、部分生僻字的完整 Unicode 字符计数需要额外知识。

5. throw new IllegalArgumentException(...) 结束当前非法调用并报告原因。赋值在检查之后，所以失败不会覆盖原姓名。

6. addHeight 接收的是增加量，不是新身高。加 2 得到172，0允许保持不变，负数被拒绝，因此没有降低身高的公开入口。

7. main 的 try 中执行可能失败的调用，catch 捕获对应异常并输出信息，使测试能够继续到最后；非法操作后对象仍是李小明、172厘米。

完整参考代码 · Main.java

```
class Person {
    private String name;
    private final String bloodType;
    private double height;

    public Person(String name, String bloodType, double height) {
        if (height < 0) {
            throw new IllegalArgumentException("身高不能为负数");
        }
        setName(name);
        this.bloodType = bloodType;
```

Main.java（续）

```
        this.height = height;
    }

    public String getName() { return name; }
    public String getBloodType() { return bloodType; }
    public double getHeight() { return height; }

    public void setName(String name) {
        if (name == null || name.length() < 2 || name.length() > 4) {
            throw new IllegalArgumentException("姓名必须为2～4个字符");
        }
        this.name = name;
    }

    public void addHeight(double increase) {
        if (increase < 0) {
            throw new IllegalArgumentException("身高不能减少");
        }
        height += increase;
    }
}

public class Main {
    public static void main(String[] args) {
        Person p = new Person("张三", "A", 170);
        System.out.println("姓名：" + p.getName() + "，血型：" + p.getBloodType()
            + "，身高：" + p.getHeight() + "厘米");
        p.setName("李小明");
        p.addHeight(2);
        System.out.println("修改后：" + p.getName() + "，身高：" + p.getHeight() + "厘米");
        try {
            p.setName("王");
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
        try {
            p.setName("一二三四五");
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
        try {
            p.addHeight(-5);
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
        System.out.println("最终：" + p.getName() + "，血型：" + p.getBloodType()
            + "，身高：" + p.getHeight() + "厘米");
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
姓名：张三，血型：A，身高：170.0厘米
修改后：李小明，身高：172.0厘米
姓名必须为2～4个字符
姓名必须为2～4个字符
身高不能减少
最终：李小明，血型：A，身高：172.0厘米
```

容易出错的地方

1. 只不写血型 setter 不够直观，使用 private final 明确不可修改。

2. 姓名范围包含2和4，判断应该是 <2 或 >4。

3. addHeight(-5) 不能先加再检查，必须先检查再改变状态。

自己动手

1. 把合法姓名分别改成2个字、4个字，确认边界也允许。

2. 调用 p.addHeight(0)，验证身高保持不变。

第 08 题 People 基类、Employee 与 Teacher 子类

原题编号：6（第二组）；本书按出现顺序编为第 8 题。

题目

原题第二次编号6：写 People 类作为基类，包含 protected String name、protected int age、People(String name,int age) 构造和 public getAge()。Employee 继承 People，新增 protected String empno；Teacher 继承 People，新增 String 类型 teano、grade。为子类编写构造并实例化测试。本答案把教师新增字段同样声明为 protected，便于展示继承结构。

基础思路

父类保存姓名和年龄。子类构造先 super(name,age) 初始化父类部分，再设置自己的编号、年级。

先掌握这些知识

继承 extends、protected、super 构造调用、继承方法

一步一步理解

1. Employee extends People 表示员工是一种人，Teacher extends People 表示教师也是一种人。两个子类直接继承 People，不是 Teacher 继承 Employee。

2. protected 成员在同包和子类等规定范围可见，所以子类的 showInfo 可以直接读取父类 name；它并不是对所有外部代码公开。

3. 子类对象既有父类定义的姓名、年龄，也有自己的编号；不需要在子类重复声明 name 和 age。

4. super(name, age) 调用父类构造，必须作为子类构造第一条语句。父类没有无参构造，因此这一行不可省略。

5. 构造方法不被继承，所以两个子类都编写自己的构造。this.empno = empno 初始化员工部分。

6. getAge 是父类公开方法，子类直接继承使用；showInfo 中 getAge() 就返回当前对象继承的年龄。

7. Java 的字符串类型写 String，原题的 string 应按 Java 正确大小写修正。

完整参考代码 · Main.java

```
class People {
    protected String name;
    protected int age;

    public People(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public int getAge() { return age; }
}

class Employee extends People {
    protected String empno;

    public Employee(String name, int age, String empno) {
        super(name, age);
        this.empno = empno;
    }

    public void showInfo() {
        System.out.println("员工：" + name + "，年龄：" + getAge() + "，工号：" + empno);
    }
}

class Teacher extends People {
    protected String teano;
    protected String grade;

    public Teacher(String name, int age, String teano, String grade) {
        super(name, age);
        this.teano = teano;
        this.grade = grade;
    }

    public void showInfo() {
        System.out.println("教师：" + name + "，年龄：" + getAge()
            + "，教师号：" + teano + "，年级：" + grade);
    }
}

public class Main {
    public static void main(String[] args) {
        Employee e = new Employee("张三", 25, "E001");
        Teacher t = new Teacher("李四", 35, "T001", "大二");
        e.showInfo();
        t.showInfo();
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
员工：张三，年龄：25，工号：E001
教师：李四，年龄：35，教师号：T001，年级：大二
```

容易出错的地方

1. 不能写 class Employee implements People，类继承类要用 extends。
2. People 只有有参构造时，子类构造不能依赖自动的 super()。
3. Employee 和后面的工资题同名，但各题独立编译，不要合并到同一目录。

自己动手

1. 再创建第二名教师，使用不同教师号和年级。
2. 在 main 直接调用 e.getAge()，观察子类怎样使用继承方法。

第 09 题 Animal、Cat、Bird 与运行时多态

原题编号：7（第二组）；本书按出现顺序编为第 9 题。

题目

原题第二次编号7：定义 Animal，包含 name、构造和 sing()；Cat 继承它，新增 furColor、构造并重写 sing()；Bird 继承它，新增 featherColor、构造并重写 sing()。main 输出“1.Cat 2.Bird”，从标准输入读取1或2，创建相应对象并赋给 Animal 类型变量 pet，最后调用 pet.sing()。原题未指定名字和颜色输入，因此示例固定名字、颜色，仅由用户选择种类。

基础思路

变量声明为 Animal，按输入建立 Cat 或 Bird，再统一调用 sing，Java 会执行实际对象所属子类的方法。

先掌握这些知识

方法重写、多态、父类引用、Scanner、if/else

一步一步理解

1. 先写父类 Animal 保存共同信息 name 和共同方法 sing，再由 Cat、Bird 增加不同颜色字段。
2. 子类构造使用 super(name) 初始化父类字段，随后保存各自毛色或羽色。
3. @Override 下的方法签名与父类 sing() 相同，所以这是重写。Cat 和 Bird 各自给出适合本类的叫声。
4. Animal pet 只声明一个父类类型的引用变量，尚未创建对象；执行某个 if 分支中的 new 才创建真正的猫或鸟。
5. 父类引用可以指向子类对象，例如 Animal pet = new Cat(...)，这叫向上转型，通常不需要显式转换。

6. 调用 pet.sing() 时由对象实际类型决定执行哪个重写版本：输入1调用猫，输入2调用鸟，这就是运行时多态。

7. 非法选择分支执行 return 结束 main，保证后面的 pet.sing() 不会使用尚未赋值的 pet。

完整参考代码 · Main.java

```
import java.util.Scanner;

class Animal {
    protected String name;

    public Animal(String name) { this.name = name; }

    public void sing() {
        System.out.println(name + "在叫");
    }
}

class Cat extends Animal {
    private String furColor;

    public Cat(String name, String furColor) {
        super(name);
        this.furColor = furColor;
    }

    @Override
    public void sing() {
        System.out.println("猫" + name + "，毛色：" + furColor + "，喵喵喵");
    }
}

class Bird extends Animal {
    private String featherColor;

    public Bird(String name, String featherColor) {
        super(name);
        this.featherColor = featherColor;
    }

    @Override
    public void sing() {
        System.out.println("鸟" + name + "，羽毛颜色：" + featherColor + "，叽叽喳喳");
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("1.Cat  2.Bird");
        int choice = sc.nextInt();
        Animal pet;
        if (choice == 1) {
            pet = new Cat("小花", "白色");
        } else if (choice == 2) {
            pet = new Bird("小黄", "黄色");
        } else {
            System.out.println("请选择1或2");
```

Main.java（续）

```
                return;
            }
            pet.sing();
        }
    }
```

样例输入

```
1
```

样例输出

```
1.Cat  2.Bird
猫小花，毛色：白色，喵喵喵
```

容易出错的地方

1. pet 的声明类型虽然是 Animal，也不表示一定执行父类 sing。
2. 方法名、参数列表要一致，sing(String s) 不会重写 sing()。
3. 输入必须是整数；非数字输入的容错不属于本基础示例。

自己动手

1. 分别输入1、2，比较同一行 pet.sing() 的输出。
2. 新增 Dog 子类，并为菜单添加第3种选择。

第 10 题 Vehicle 接口、Bike/Bus 与 Driver

原题编号：8；本书按出现顺序编为第 10 题。

题目

创建 Vehicle 接口，声明 start、stop 两个“各带一个参数”的方法；Bike、Bus 实现接口。Driver 类定义 void drive(Vehicle v)，main 创建 Bike、Bus、Driver，分别传入 drive。原文同时把方法写作 start()/stop()，与“一个参数”矛盾；本答案优先满足文字要求，统一声明 start(String driverName)、stop(String driverName)，参数表示驾驶者，Driver.drive(Vehicle v) 的签名保持原要求。

基础思路

接口规定启动和停止的共同调用方式；两种交通工具分别实现，Driver 只接收 Vehicle 并依次调用两个行为。

先掌握这些知识

接口 interface、implements、接口参数、多态协作

一步一步理解

1. interface Vehicle 写出车辆必须具备的行为约定。这里的方法没有方法体，以分号结尾。

2. 接口中这两种普通抽象方法默认是 public abstract；实现类必须使用 public，不能缩小可见范围。

3. Bike implements Vehicle 表示自行车承诺实现接口规定的所有方法，Bus 也作同样承诺。

4. start 和 stop 各接收一个 String 参数，分别在启动和停止时显示驾驶者，明确满足原题“各带一个参数”。

5. drive(Vehicle v) 中 v 可引用任意实现 Vehicle 的对象，因此传 Bike 和 Bus 都合法。

6. v.start("司机") 的具体输出由当前对象决定；Driver 不需要分别写 drive(Bike) 和 drive(Bus)。

7. 原题括号写成空参数但文字要求一个参数，这里作了显式约定；如果课堂确认采用无参版，应同步删除接口、实现和调用中的参数。

完整参考代码 · Main.java

```
interface Vehicle {
    void start(String driverName);
    void stop(String driverName);
}

class Bike implements Vehicle {
    @Override
    public void start(String driverName) {
        System.out.println(driverName + "启动自行车");
    }

    @Override
    public void stop(String driverName) {
        System.out.println(driverName + "停止自行车");
    }
}

class Bus implements Vehicle {
    @Override
    public void start(String driverName) {
        System.out.println(driverName + "启动公交车");
    }

    @Override
    public void stop(String driverName) {
        System.out.println(driverName + "停止公交车");
    }
}

class Driver {
    public void drive(Vehicle v) {
        v.start("司机");
        v.stop("司机");
    }
}

public class Main {
    public static void main(String[] args) {
        Bike bike = new Bike();
        Bus bus = new Bus();
        Driver driver = new Driver();
        driver.drive(bike);
```

Main.java（续）

```
        driver.drive(bus);
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
司机启动自行车
司机停止自行车
司机启动公交车
司机停止公交车
```

容易出错的地方

1. 实现接口用 implements，不能写 extends Vehicle 来声明普通实现类。
2. 接口声明和两个实现的方法参数必须完全一致。
3. 不能 new Vehicle()，应创建具体 Bike 或 Bus 对象。

自己动手

1. 在 drive 中把“司机”改成自己的姓名。
2. 新增 Car 实现同一接口，并复用现有 Driver.drive。

第 11 题 Circle 与 Cylinder：继承、面积重写和体积

原题编号：9；本书按出现顺序编为第 11 题。

题目

Circle 包含私有 double Radius，无参构造将半径设为0，有参 Circle(double r) 设置半径；提供 getArea 和 getPerimeter。Cylinder 继承 Circle，增加私有 double height，通过 Cylinder(double r,double h) 设置半径与高，提供 getVolume 并重写 getArea 求圆柱表面积。main 创建半径3、高5的圆柱，分别显示底面圆面积、圆周长、圆柱体积和表面积。原题未指定输入，本例用构造参数设置。

基础思路

圆面积 πr^2，周长$2\pi r$；圆柱体积=底面积×高，表面积=2×底面积+周长×高。子类用 super.getArea() 取得底面积，避免把重写后的表面积误当底面积。

先掌握这些知识

继承、super、方法重写、动态派发、Math.PI、格式化输出

一步一步理解

1. Circle 用 Radius 保存半径，按题面保留大写字段名；一般 Java 习惯会写 radius。double 是浮点类型，Math.PI 提供圆周率。

2. Circle() 调用 this(0) 建立半径0的圆。Cylinder(3,5) 先执行 super(3) 设置父类私有半径，再设置自己的高度5。

3. Radius 虽然是 private，Cylinder 仍可通过父类公开方法使用它，不能直接访问 Radius。

4. Cylinder 重写 getArea 后，c.getArea() 的含义是圆柱表面积。仅把 c 转成 Circle 引用仍会执行子类版本，不能用强制转换拿到底面积。

5. super.getArea() 明确调用父类圆面积实现。getBaseArea 用它把底面积公开出来，getVolume 用它计算 $\pi r^2 h$，不能误写 getArea()*height。

6. 表面积为两个底面加侧面，侧面展开为长方形，长为圆周长$2\pi r$，宽为高度h，因此公式是$2\pi r^2+2\pi rh$。

7. 半径3、高5时，底面积$9\pi \approx 28.27$，周长$6\pi \approx 18.85$，体积$45\pi \approx 141.37$，表面积$48\pi \approx 150.80$。

8. printf 中 %.2f 表示保留两位小数显示，%n 换行；Locale.ROOT 固定使用小数点，计算本身没有提前四舍五入。

完整参考代码 · Main.java

```
import java.util.Locale;

class Circle {
    private double Radius;

    public Circle() { this(0); }

    public Circle(double r) {
        this.Radius = r;
    }

    public double getArea() {
        return Math.PI * Radius * Radius;
    }

    public double getPerimeter() {
        return 2 * Math.PI * Radius;
    }
}

class Cylinder extends Circle {
    private double height;

    public Cylinder(double r, double h) {
        super(r);
        this.height = h;
    }

    public double getBaseArea() {
        return super.getArea();
    }

    public double getVolume() {
        return super.getArea() * height;
```

Main.java（续）

```
    }

    @Override
    public double getArea() {
        return 2 * super.getArea() + super.getPerimeter() * height;
    }
}

public class Main {
    public static void main(String[] args) {
        Cylinder c = new Cylinder(3, 5);
        System.out.printf(Locale.ROOT, "圆面积：%.2f%n", c.getBaseArea());
        System.out.printf(Locale.ROOT, "圆周长：%.2f%n", c.getPerimeter());
        System.out.printf(Locale.ROOT, "圆柱体积：%.2f%n", c.getVolume());
        System.out.printf(Locale.ROOT, "圆柱表面积：%.2f%n", c.getArea());
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
圆面积：28.27
圆周长：18.85
圆柱体积：141.37
圆柱表面积：150.80
```

容易出错的地方

1. getVolume 不能使用重写后的 getArea 当底面积，否则得到表面积乘高度。
2. 在 Cylinder.getArea 中调用自身 getArea 会无限递归，应该用 super.getArea。
3. 题目要求圆柱表面积，不是仅求侧面积；必须加两个底面。

自己动手

1. 把半径设为1、高设为2，手算体积2π、表面积6π。
2. 创建 Circle()，观察半径0时面积和周长都为0。

第 12 题 CanCry 接口与 G.makeCry 测试

原题编号：10；本书按出现顺序编为第 12 题。

题目

定义接口 CanCry，包含 public void cry()；Dog、Cat 实现接口，分别准确输出“我是狗，我的叫声是汪汪汪”“我是猫，我的叫声是喵喵喵”。定义 G 类和 void makeCry(CanCry c)，在 G.main 内创建 dog、cat、g 并调用 g.makeCry。原题 makecry 与 makeCry 大小写不一致，这里统一为 makeCry。为在线练习统一 Main.java，额外增加 public Main 作为入口，委托给保留的 G.main。

基础思路

CanCry 统一叫声接口，makeCry 只调用 c.cry。保留题面 G 类与测试对象，Main 只负责启动 G.main。

先掌握这些知识

接口实现、接口作为参数、多态、程序入口

一步一步理解

1. CanCry 表示“具有叫的能力”，不必限定调用者一定是某个 Animal 子类；凡是实现它的类都能传给 makeCry。

2. 接口中的 cry 只有方法声明，Dog 和 Cat 分别用 public void cry() 给出具体方法体。

3. makeCry(CanCry c) 的参数使用接口类型，c.cry() 在运行时根据实际对象调用狗或猫的实现。

4. G g = new G() 创建测试类对象，因为 makeCry 不是 static，需要通过 g 调用。

5. g.makeCry(dog) 和 g.makeCry(cat) 调用同一方法，实参类型不同仍然可行，因为它们都实现了 CanCry。

6. 原题要求的 G 类、G.main 以及 dog/cat/g 均保留；外加 public Main.main 调用 G.main 是为了所有练习统一保存为 Main.java。若单独交作业，亦可删 Main 并把 G 改 public，保存为 G.java。

完整参考代码 · Main.java

```
interface CanCry {
    public void cry();
}

class Dog implements CanCry {
    @Override
    public void cry() {
        System.out.println("我是狗，我的叫声是汪汪汪");
    }
}

class Cat implements CanCry {
    @Override
    public void cry() {
        System.out.println("我是猫，我的叫声是喵喵喵");
    }
}

class G {
    public void makeCry(CanCry c) {
```

Main.java（续）

```
        c.cry();
    }

    public static void main(String[] args) {
        Dog dog = new Dog();
        Cat cat = new Cat();
        G g = new G();
        g.makeCry(dog);
        g.makeCry(cat);
    }
}

public class Main {
    public static void main(String[] args) {
        G.main(args);
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
我是狗，我的叫声是汪汪汪
我是猫，我的叫声是喵喵喵
```

容易出错的地方

1. Java 大小写敏感，定义 makeCry 后不能调用 makecry。
2. CanCry 的实现 cry 必须是 public。
3. 一个文件只能有一个 public 顶层类，这里仅 Main 为 public。

自己动手

1. 交换两次 makeCry 调用的顺序。
2. 新增一个实现 CanCry 的 Duck 类，让 g.makeCry 接收它。

第 13 题 抽象 Shape 与圆、矩形面积

原题编号：11；本书按出现顺序编为第 13 题。

题目

编写 abstract Shape，声明抽象 getArea()；定义 Circle 与 Rectangle 继承 Shape，分别用圆和矩形公式实现 getArea。main 创建两个具体图形并输出面积，展示父类类型数组统一处理不同图形。原题未指定尺寸，本例圆半径3，矩形长4宽5。

基础思路

Shape 只规定面积方法，由子类分别实现。将两个对象放进 Shape[]，循环时统一调用 getArea。

先掌握这些知识

abstract 抽象类、抽象方法、方法重写、多态数组

一步一步理解

1. abstract class Shape 表示抽象图形，不能直接 new Shape()，因为尚未说明具体图形怎么算面积。

2. public abstract double getArea(); 只有声明没有方法体，要求所有具体子类给出可运行实现。

3. Circle extends Shape 继承面积约定，用 π r² 实现；Rectangle 用长×宽实现。返回类型都保持 double。

4. Shape[] 是可以保存 Shape 引用的数组，圆和矩形都属于 Shape，所以可一起存放。

5. 数组下标从0开始，shapes.length 是元素个数2。循环条件 i < shapes.length 使下标只取0和1。

6. shapes[i].getArea() 的写法相同，但分别执行 Circle 和 Rectangle 的重写方法；这将抽象类和多态结合起来。

7. names 数组只为给输出添加对应标签。%s 显示文字，%.2f 显示两位小数。

完整参考代码 · Main.java

```
import java.util.Locale;

abstract class Shape {
    public abstract double getArea();
}

class Circle extends Shape {
    private double radius;

    public Circle(double radius) { this.radius = radius; }

    @Override
    public double getArea() {
        return Math.PI * radius * radius;
    }
}

class Rectangle extends Shape {
    private double length;
    private double width;

    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    @Override
    public double getArea() {
        return length * width;
    }
}
```

Main.java（续）

```
public class Main {
    public static void main(String[] args) {
        Shape[] shapes = {new Circle(3), new Rectangle(4, 5)};
        String[] names = {"圆面积", "矩形面积"};
        for (int i = 0; i < shapes.length; i++) {
            System.out.printf(Locale.ROOT, "%s：%.2f%n", names[i], shapes[i].getArea());
        }
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
圆面积：28.27
矩形面积：20.00
```

容易出错的地方

1. 抽象方法结尾是分号，不写空大括号冒充抽象实现。
2. 非抽象子类必须实现继承的全部抽象方法。
3. 不要写 i <= shapes.length，最大合法下标是 length-1。

自己动手

1. 新增 Triangle extends Shape，给出底和高并返回底×高÷2。
2. 把圆和矩形换到数组中的相反顺序，并同步调整 names。

第 14 题 员工数组、平均工资项目与应发工资

原题编号：12；本书按出现顺序编为第 14 题。

题目

设计 Employee，字段 name、age、basic、allowance、tax；构造初始化全部字段；total() 返回 basic+allowance-tax。main 使用 Employee[] 保存原表4名员工：孙悟空/22/3210/1240/280，猪八戒/21/1820/980/160，沙和尚/28/2100/1020/210，白骨精/18/2560/1320/245。计算平均 basic、平均 allowance，再输出所有员工基本信息与 total() 结果。按原题把扣税后的结果称为“应发总工资”。

基础思路

用对象数组存4条记录，第一次循环累加基本工资和津贴，除以人数求平均；第二次循环输出每名员工及其工资。

先掌握这些知识

对象数组、for 循环、累加器、平均值、实例方法

一步一步理解

1. 每个 Employee 保存完整一行数据，避免把姓名和工资拆成互相难对应的多组变量；数组有4个对象。

2. 构造参数严格对应姓名、年龄、基本工资、津贴、扣税；实例化顺序应与原表一致。

3. sumBasic 和 sumAllowance 是累加器，循环前置0，每次把当前员工的一项工资加进去。

4. 基本工资总和3210+1820+2100+2560=9690，平均9690÷4=2422.5；津贴总和4560，平均1140。

5. 累加器是 double，除以 int 人数时会进行浮点除法，不会把2422.5截成2422。

6. total 是当前员工的方法，读取当前对象的 basic、allowance、tax，例如孙悟空3210+1240-280=4170。

7. 第二轮循环调用 showInfo 输出全部5项资料以及 total 的计算值。按题面命名为“应发总工资”；日常工资术语中扣税后也常称实发。

8. %.2f 使金额都显示两位小数，便于逐行核对；这里使用 double 是基础练习范围内的写法。

完整参考代码 · Main.java

```
import java.util.Locale;

class Employee {
    private String name;
    private int age;
    private double basic;
    private double allowance;
    private double tax;

    public Employee(String name, int age, double basic, double allowance, double tax) {
        this.name = name;
        this.age = age;
        this.basic = basic;
        this.allowance = allowance;
        this.tax = tax;
    }

    public double getBasic() { return basic; }
    public double getAllowance() { return allowance; }

    public double total() {
        return basic + allowance - tax;
    }

    public void showInfo() {
        System.out.printf(Locale.ROOT,
            "%s 年龄%d 基本工资%.2f 津贴%.2f 扣税%.2f 应发总工资%.2f%n",
            name, age, basic, allowance, tax, total());
    }
}

public class Main {
    public static void main(String[] args) {
        Employee[] employees = {
            new Employee("孙悟空", 22, 3210, 1240, 280),
```

Main.java（续）

```
            new Employee("猪八戒", 21, 1820, 980, 160),
            new Employee("沙和尚", 28, 2100, 1020, 210),
            new Employee("白骨精", 18, 2560, 1320, 245)
        };
        double sumBasic = 0;
        double sumAllowance = 0;
        for (int i = 0; i < employees.length; i++) {
            sumBasic += employees[i].getBasic();
            sumAllowance += employees[i].getAllowance();
        }
        System.out.printf(Locale.ROOT, "平均基本工资：%.2f%n", sumBasic / employees.length);
        System.out.printf(Locale.ROOT, "平均津贴：%.2f%n", sumAllowance / employees.length);
        for (int i = 0; i < employees.length; i++) {
            employees[i].showInfo();
        }
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
平均基本工资：2422.50
平均津贴：1140.00
孙悟空 年龄22 基本工资3210.00 津贴1240.00 扣税280.00 应发总工资4170.00
猪八戒 年龄21 基本工资1820.00 津贴980.00 扣税160.00 应发总工资2640.00
沙和尚 年龄28 基本工资2100.00 津贴1020.00 扣税210.00 应发总工资2910.00
白骨精 年龄18 基本工资2560.00 津贴1320.00 扣税245.00 应发总工资3635.00
```

容易出错的地方

1. 先求所有人的总和再除以人数，不能在循环中反复除以人数。
2. total 公式必须减去 tax，不能加上扣税。
3. 统计字段要分别累加，平均津贴不是平均应发工资。

自己动手

1. 手算4个人的 total 并与程序逐个核对。
2. 增加第5名员工，使用 employees.length 验证平均值自动按5人计算。

第 15 题 Graph 类图：三种图形与 Client 绘制

原题编号：13；本书按出现顺序编为第 15 题。

题目

按原PDF第5页顶部类图实现：Graph 接口声明 public drawing():void；Rectangle 含 private double length、width 和 Rectangle(double length,double width)；Circle 含 private double radius 和 Circle(double radius)；Triangle 含 private double a、b、c 和 Triangle(double a,double b,double c)。三者实现 Graph。Client 包含 public myDrawing(Graph graph):void，根据实际对象调用 drawing。main 创建三种图形并测试；本控制台题以输出形状名称和尺寸表示绘制，原题未要求图形窗口。

基础思路

先把图里的字段和构造逐项转成Java，再让三类实现 drawing；Client 只依赖 Graph，并将绘制工作交给传入对象。

先掌握这些知识

UML 类图、接口、依赖关系、构造参数、多态

一步一步理解

1. 先读类图：<<Interface>> 表示接口；字段名后的 double 是类型；方法名后括号表示参数，末尾 void 表示无返回值。

2. 类图中三种具体图形都指向 Graph 的接口实现关系，翻译为 implements Graph；不是 Circle 继承 Rectangle。

3. Rectangle 保存长宽，Circle 保存半径，Triangle 按图保存三条边 a、b、c。三角形不能擅自改成底和高而漏掉原图字段。

4. 每个类都要有图上规定的有参构造；this.a = a 这种语句把传入尺寸保存到当前对象。

5. Graph 只声明 drawing，三种图形分别实现它。本题没有窗口要求，所以输出形状和尺寸演示“绘制”行为。

6. Client.myDrawing(Graph graph) 的参数体现类图中的依赖：Client 使用接口，不需要了解具体图形的字段。

7. graph.drawing() 自动选择实际对象的方法。main 连续传矩形、圆、三角形，展示一个客户端调用三种实现。

完整参考代码 · Main.java

```
interface Graph {
    void drawing();
}

class Rectangle implements Graph {
    private double length;
    private double width;
```

Main.java（续）

```
    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    @Override
    public void drawing() {
        System.out.println("绘制矩形：长=" + length + ", 宽=" + width);
    }
}

class Circle implements Graph {
    private double radius;

    public Circle(double radius) { this.radius = radius; }

    @Override
    public void drawing() {
        System.out.println("绘制圆：半径=" + radius);
    }
}

class Triangle implements Graph {
    private double a;
    private double b;
    private double c;

    public Triangle(double a, double b, double c) {
        this.a = a;
        this.b = b;
        this.c = c;
    }

    @Override
    public void drawing() {
        System.out.println("绘制三角形：三边=" + a + "、" + b + "、" + c);
    }
}

class Client {
    public void myDrawing(Graph graph) {
        graph.drawing();
    }
}

public class Main {
    public static void main(String[] args) {
        Client client = new Client();
        client.myDrawing(new Rectangle(4, 3));
        client.myDrawing(new Circle(2));
        client.myDrawing(new Triangle(3, 4, 5));
    }
}
```

样例输入

（无输入，直接运行）

样例输出

```
绘制矩形：长=4.0，宽=3.0
绘制圆：半径=2.0
绘制三角形：三边=3.0、4.0、5.0
```

容易出错的地方

1. 必须保留 Triangle 的 a、b、c 和三参数构造，不能只创建空类。

2. 方法名按图写 myDrawing 和 drawing，大小写不能随意修改。

3. 三角形示例3、4、5合法；若允许任意输入，应增加三边均正和两边之和大于第三边的校验。

自己动手

1. 增加一个 Square implements Graph，让现有 Client 直接绘制它。

2. 把 Triangle 改成其他合法三边，核对输出字段是否来自构造参数。

第 16 题 Animal、Ibehavior 与 Lion 类图实现

原题编号：14；本书按出现顺序编为第 16 题。

题目

按原PDF第5页图1实现：Animal 具有 public height:double、weight:Double、bodyColor:String；Ibehavior 接口声明 walking() 和 eating(String food)；Lion 继承 Animal，同时实现 Ibehavior，新增 public toothNumber:int、tailLength:double。测试输出狮子特征，再调用 walking、eating。按图2示例取体重56.0、体高0.6、皮色黄色、牙齿38、尾长1.2；输出“狮子在奔跑！”和“狮子在吃野牛”。

基础思路

Animal 保存共同特征，Lion 新增狮子特征；用 extends Animal implements Ibehavior 同时表达继承和行为实现，打印原图示例。

先掌握这些知识

继承与接口组合、UML、包装类型 Double、对象特征、行为方法

一步一步理解

1. 图中 + 表示 public，因此本题按图将 Animal 和 Lion 的字段写为 public。封装练习常用 private，但这里优先遵照类图。

2. height 是基本类型 double，weight 在原图是大写 Double，即包装类型；传入56.0时Java自动把 double 包装成 Double。

3. extends Animal 表明狮子是一种动物，继承 height、weight、bodyColor；implements Ibehavior 表明它实现接口规定的行为。

4. Java 中一个类只能直接继承一个类，但可以同时实现接口。关键字次序为 extends 在前，implements 在后。

5. 构造方法中的 super(...) 把共有特征交给父类初始化，随后初始化牙齿数和尾长。构造参数顺序先体高再体重，不能颠倒。

6. walking 没有参数，按原图示例输出“狮子在奔跑！”；eating(String food) 有一个字符串参数，因此可把“野牛”替换成其他食物。

7. showInfo 是为输出特征补充的方法，并不改变原类图关系。整个测试验证了字段、继承关系和接口行为。

完整参考代码 · Main.java

```
class Animal {
    public double height;
    public Double weight;
    public String bodyColor;

    public Animal(double height, Double weight, String bodyColor) {
        this.height = height;
        this.weight = weight;
        this.bodyColor = bodyColor;
    }
}

interface Ibehavior {
    void walking();
    void eating(String food);
}

class Lion extends Animal implements Ibehavior {
    public int toothNumber;
    public double tailLength;

    public Lion(double height, Double weight, String bodyColor,
                int toothNumber, double tailLength) {
        super(height, weight, bodyColor);
        this.toothNumber = toothNumber;
        this.tailLength = tailLength;
    }

    public void showInfo() {
        System.out.println("狮子：");
        System.out.println("体重：" + weight + "，体高：" + height + "，皮色："
            + bodyColor + "，牙齿数：" + toothNumber + "，尾巴长度：" + tailLength);
    }

    @Override
    public void walking() {
        System.out.println("狮子在奔跑！");
    }

    @Override
    public void eating(String food) {
        System.out.println("狮子在吃" + food);
    }
}

public class Main {
    public static void main(String[] args) {
        Lion lion = new Lion(0.6, 56.0, "黄色", 38, 1.2);
        lion.showInfo();
```

Main.java（续）

```
        lion.walking();
        lion.eating("野牛");
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
狮子：
体重：56.0，体高：0.6，皮色：黄色，牙齿数：38，尾巴长度：1.2
狮子在奔跑！
狮子在吃野牛
```

容易出错的地方

1. 不能因为 walking 中文是行走就漏掉原图给出的“狮子在奔跑！”示例。
2. Double 和 double 大小写不同，本题按图保留 Double。
3. 类名、接口名和实现方法要统一，原题接口写作 Ibehavior。

自己动手

1. 把食物参数改为“肉”，观察 eating 方法复用。
2. 使用 Ibehavior behavior = lion，再调用两个行为，体会接口引用。

第 17 题 网络用户类与默认邮箱构造重载

原题编号：15；本书按出现顺序编为第 17 题。

题目

定义网络用户类，保存用户ID、密码、email。建立实例时必须提供ID和密码，email可省略；提供email时原样使用，省略时邮箱为用户ID拼接“@gameschool.com”。Java不支持给普通形参直接写默认值，因此提供三参数构造和二参数重载构造；不提供无参构造。main 分别验证自定义邮箱和默认邮箱。

基础思路

三参数构造保存完整信息，二参数构造调用 this(userId,password,userId+"@gameschool.com")，复用初始化过程。

先掌握这些知识

构造重载、this(...)、默认参数的替代、字符串拼接、封装

一步一步理解

1. NetworkUser 的三个 private 字段分别保存ID、密码、邮箱；对象对外只显示ID和邮箱，密码不打印。

2. 有三个实参时匹配三参数构造，例如自定义邮箱 zhangsan@example.com；有两个实参时匹配二参数构造。

3. 二参数构造省略的是邮箱，ID和密码在两个构造中始终必填。我们没有声明无参构造，因此不能 new NetworkUser()。

4. userId + "@gameschool.com" 进行字符串连接，例如 lisi 得到 lisi@gameschool.com。

5. this(...) 表示调用本类另一个构造，必须放在当前构造第一条语句；这与 this.userId 访问当前字段的用法不同。

6. 原题说“缺省”是省略第三个参数，不自动等价于给第三参数传null或空字符串；如果传三个参数，本答案按显式值保存。

7. 密码字段满足课堂对象建模要求。实际登录系统需要额外的密码散列与安全设计，本例不把它当作可上线的账户系统。

完整参考代码 · Main.java

```
class NetworkUser {
    private String userId;
    private String password;
    private String email;

    public NetworkUser(String userId, String password, String email) {
        this.userId = userId;
        this.password = password;
        this.email = email;
    }

    public NetworkUser(String userId, String password) {
        this(userId, password, userId + "@gameschool.com");
    }

    public String getUserId() { return userId; }
    public String getEmail() { return email; }

    public void showInfo() {
        System.out.println("用户ID: " + userId + ", 邮箱: " + email);
    }
}

public class Main {
    public static void main(String[] args) {
        NetworkUser u1 = new NetworkUser("zhangsan", "123456", "zhangsan@example.com");
        NetworkUser u2 = new NetworkUser("lisi", "abcdef");
        u1.showInfo();
        u2.showInfo();
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
用户ID：zhangsan，邮箱：zhangsan@example.com
用户ID：lisi，邮箱：lisi@gameschool.com
```

容易出错的地方

1. Java不支持 NetworkUser(String id, String pwd, String email = ...) 这种形参默认值语法。
2. this(...) 前不能先执行其他语句。
3. 默认邮箱应使用当前用户ID，不能写死成某个固定名字。

自己动手

1. 创建用户 wangwu 且省略邮箱，先猜测再验证邮箱。
2. 增加一个 changeEmail(String email) 方法，通过它修改第二名用户的邮箱。

复习与参考

复习时先做没有输入的题，再做单次输入，最后做循环输入和多个对象的题。遇到相同类名，回到对应题目录单独编译。

原题库：用户提供的阶段一、阶段二题库 PDF。原题中的重复编号、示例编号和歧义已经在封面说明或对应题目中标出。

Oracle Java Tutorials：语言基础

https://docs.oracle.com/javase/tutorial/java/nutsandbolts/index.html

Oracle Java Tutorials：面向对象概念

https://docs.oracle.com/javase/tutorial/java/concepts/index.html

本文使用 JDK 17 可运行的基础语法；上面的概念教程版本较早，变量、循环、类与继承等基础仍适用。